Tutorial · Verificación end-to-end de LaPlacita desplegada

> **Propósito:** procedimiento reproducible para verificar la aplicación completa en los entornos desplegados — del pre-pedido en el teléfono a la validación del PIN en el backend. Escrito para la demo del 14/10/2026; reutilizable como rutina de verificación de cualquier despliegue nuevo.

Entornos

Rol	URL	Nota
Frontend (PWA del usuario)	`https://laplacita-app.iscoutb.dev`	Dokploy · HTTPS (habilita la instalación de la PWA)
Backend de pruebas (con BD real)	`https://laplacita-test.iscoutb.dev`	Dokploy · PostgreSQL de ADR-0014 · **usar para las operaciones**
Backend de producción	`https://laplacita-app.graymoss-fdd72159.canadacentral.azurecontainerapps.io`	Azure · modo memoria hasta que el firewall habilite la BD (pendiente documentado)
Contrato OpenAPI (Swagger)	`https://laplacita-test.iscoutb.dev/docs`	Misma especificación que valida la prueba de contrato

Parte 0 — Preparación (10 min)

En el celular:

1. Abrir `https://laplacita-app.iscoutb.dev` en Chrome (Android) o Safari (iOS).
2. **Instalar la PWA**: Android → menú ⋮ → *Instalar aplicación*; iOS → Compartir → *Añadir a pantalla de inicio*.
3. Probar el dominio **desde el WiFi del campus y desde datos móviles** (si el WiFi del campus bloquea el dominio, la demo se hace por hotspot del teléfono).

En el portátil: 4. Abrir el Swagger del backend de pruebas. 5. Verificar `GET https://laplacita-test.iscoutb.dev/api/v1/health` → `{"status":"ok"}`.

Parte 1 — El cliente pide (celular)

1. Onboarding: nombre → *Empezar a pedir* (perfil local del dispositivo; el inicio de sesión con credenciales llega con la identidad de tenant, corte 3).
2. Seleccionar **El Mesón** → el menú se consulta del backend (`GET /api/v1/catalogo/tiendas/tienda-01/productos`).

3. Elegir un producto, tocar **+** (el contador arranca en 0 por diseño) → el botón muestra el total → **Pedir**. Agregar un segundo producto.
4. Abrir el **carrito flotante**: productos agrupados por tienda, cantidades y total.
5. **Hacer pedido** → el contrato v1 crea **un pedido por producto** (`POST /pedidos`) → navegación a *Mis pedidos* con los pedidos creados y su estado.
6. Entrar al **seguimiento** de un pedido: línea de tiempo de 4 estados, actualización automática cada 3 s, código de retiro (`pedidoId`).

> El PIN no viaja por la lectura de pedido (ADR-0013, A-06): el usuario presenta su `pedidoId` en ventanilla; el PIN lo emite el establecimiento al marcar *Listo*.

Parte 2 — El establecimiento atiende (Swagger del backend)

Con el `pedidoId` visible en el teléfono:

Paso	Operación (Try it out)	Resultado en el teléfono (≤3 s)
1	`POST /pagos/{pedidoId}/confirmar` · `tiendaId: tienda-01`	Estado → **En preparación** 👨‍🍳
2	`POST /entrega/{pedidoId}/listo?tiendaId=tienda-01` — la respuesta entrega el **PIN** de 4 dígitos	Estado → **Listo** 🔔
3	`POST /entrega/{pedidoId}/validar` · body `{"pinIngresado":"1234"}`	Estado → **Entregado** ✅

Pruebas negativas (A-06 / ESC-03):

- Validar con PIN incorrecto → `PIN incorrecto` + `pinesRechazados` sube en métricas.
- Revalidar un pedido ya Entregado → rechazo por estado (el PIN no se reutiliza).
- Consultar con otra tienda → `Pedido … no encontrado en la tienda …` (aislamiento RES-05).

Parte 3 — Persistencia ante reinicios (ESC-01 · verificación estrella)

1. Crear 2-3 pedidos desde el celular y dejarlos en `Recibido` (anotar ids en *Mis pedidos*).
2. Panel de Dokploy → app del **backend de pruebas** → **Restart** (o Redeploy) → esperar ~30 s.
3. Volver a *Mis pedidos* / seguimiento → los pedidos **sobreviven con su estado** (`GET /pedidos/{id}` sigue respondiendo).

Criterio: ESC-01 — 0 pedidos perdidos tras el reinicio. Medición documentada con horas en `evidencias-s10.md` §6 (3/3 supervivientes).

Parte 4 — Observabilidad y cierre

- `GET /api/v1/metricas` → contadores en vivo: `pedidosCreados`, `pagosConfirmados`, `pedidosListos`, `entregasValidadas`, `pinesRechazados`, `pinesBloqueados`.
- El Swagger (`/docs`) evidencia el contrato OpenAPI 3.1 versionado (v1) que valida la prueba de contrato en CI.
- El menú mostrado es el del backend: la app no inventa datos ni endpoints.

Solución de problemas

Síntoma	Causa probable	Acción
El menú no carga en el teléfono	Red del campus bloquea el dominio	Datos móviles / hotspot
El carrito se queda en "Creando pedidos…"	`BACKEND_URL` del frontend mal apuntado	Dokploy → frontend → Environment → `https://laplacita-test.iscoutb.dev` → Redeploy
502 "El backend no respondió"	Backend caído o `BACKEND_URL` incorrecto	Logs del contenedor en Dokploy
"Bucle de proxy: revisa BACKEND_URL"	El frontend se proxyéa a sí mismo (env ausente)	Definir `BACKEND_URL` en Environment y Redeploy
La PWA no instala	Sin HTTPS o SW inactivo	Verificar candado del dominio; el SW solo se registra en builds de producción
Plan B total	Los entornos en la nube fallan en vivo	Demo local: `cd backend && npm run dev` + `cd frontend && npm run dev` (verificada)

Notas de alcance (deudas declaradas)

- **Sin login con credenciales:** el backend no tiene identidad de tenant aún (plan en `correcciones.md` §3, corte 3); el perfil del dispositivo es local y sin fingir seguridad.
- **Sin fotos de productos:** el contrato no expone `imageUrl` (extensión v2 pendiente); las tarjetas usan placeholders del tema.
- **Sin panel del establecimiento:** las operaciones de tienda se ejecutan con el contrato (Swagger); un panel es la siguiente iteración.
- **Notificaciones push:** la emisión de eventos (V-05) es deuda declarada; el seguimiento es por polling.